

```

=== mobile/src/components/AnimatedCheckbox.tsx ===
1  import React from 'react';
2  import { Pressable, View, StyleSheet } from 'react-native';
3  import Svg, { Path } from 'react-native-svg';
4
5  /**
6   * 动画复选框组件属性
7   */
8  interface CheckboxProps {
9    /** 是否选中 */
10   checked: boolean;
11   /** 切换选中状态的回调 */
12   onToggle: () => void;
13   /** 复选框大小 */
14   size?: number;
15   /** 复选框颜色 */
16   color?: string;
17 }
18
19 /**
20  * 动画复选框组件
21  * 提供自定义样式的复选框，包含SVG对勾
22  */
23  const AnimatedCheckbox: React.FC<CheckboxProps> = ({
24    checked,
25    onToggle,
26    size = 24,
27    color = '#80FF00',
28  }) => {
29    return (
30      <Pressable onPress={onToggle} style={styles.container}>
31        <View style={[styles.box, { width: size * 1.5, height: size * 1.5, borderColor: color }]}>
32          /* 选中时显示SVG对勾 */
33          {checked && (
34            <Svg
35              width={size * 1.5}
36              height={size * 1.5}
37              viewBox="0 0 72 97"
38              style={styles.checkmark}
39            >
40              <Path
41                d="M28.72 95.673c-6.37644-11.034-4.445-15.746-8.048-4.947-3.783-8.859-10.482-10.847-16.446-6.68
42                strokeWidth="4px"
43                stroke={color}
44                fill="none"
45                strokeLinecap="round"
46                strokeLinejoin="round"
47              />
48            </Svg>
49          )}

```

```

50   </View>
51   </Pressable>
52 );
53 };
54
55 const styles = StyleSheet.create({
56   /** 容器样式 */
57   container: {
58     justifyContent: 'center',
59     alignItems: 'center',
60   },
61   /** 复选框边框样式 */
62   box: {
63     borderWidth: 2,
64     borderRadius: 4,
65     justifyContent: 'center',
66     alignItems: 'center',
67     overflow: 'hidden',
68   },
69   /** 对勾样式 */
70   checkmark: {
71     position: 'absolute',
72   },
73 });
74
75 export default AnimatedCheckbox;
76
77 === mobile/src/components/FloatingInputBar.tsx ===
78 import React, { useRef, useEffect } from 'react';
79 import {
80   View,
81   Text,
82   TextInput,
83   TouchableOpacity,
84   Animated,
85   Keyboard,
86   Platform,
87   KeyboardEvent,
88   StyleSheet,
89   InteractionManager,
90   KeyboardTypeOptions,
91 } from 'react-native';
92 import { Ionicons } from '@expo/vector-icons';
93
94 // =====
95 // FloatingInputBar
96 // 键盘上方浮动输入栏（极简版）
97 //
98 // 点击页面上的“伪输入框”后，浮层栏出现在键盘上方并自动聚焦。
99 // 输入内容实时同步到原始字段。点「完成」收起键盘。

```

```

99 // =====
100
101 export interface InputFieldConfig {
102   id: string;
103   label: string;
104   icon: string;
105   value: string;
106   onChangeText: (text: string) => void;
107   keyboardType?: KeyboardTypeOptions;
108   secureTextEntry?: boolean;
109   placeholder?: string;
110 }
111
112 interface FloatingInputBarProps {
113   fields: InputFieldConfig[];
114   activeFieldId: string | null;
115   onDismiss: () => void;
116   colors: {
117     backgroundDark: string;
118     background: string;
119     primary: string;
120     textPrimary: string;
121     textSecondary: string;
122     borderLight: string;
123     border: string;
124   };
125 }
126
127 const BAR_BOTTOM_SPACING = 8; // 输入栏与键盘之间的间距
128
129 export const FloatingInputBar: React.FC<FloatingInputBarProps> = ({
130   fields,
131   activeFieldId,
132   onDismiss,
133   colors,
134 }) => {
135   const inputRef = useRef<TextInput>(null);
136   const translateY = useRef(new Animated.Value(0)).current;
137   const opacity = useRef(new Animated.Value(0)).current;
138
139   const activeField = fields.find((f) => f.id === activeFieldId);
140
141   // ---- 字段切换或首次挂载时聚焦 ----
142   // key={activeFieldId} 确保每次切换都创建新的 TextInput 实例
143   // autoFocus 处理首次挂载, InteractionManager 确保动画完成后聚焦
144   useEffect(() => {
145     if (activeFieldId) {
146       InteractionManager.runAfterInteractions(() => {
147         if (inputRef.current) {
148           inputRef.current.focus();

```

```

149     }
150   });
151 }
152 }, [activeFieldId]);
153
154 // ---- 键盘动画 ----
155 useEffect(() => {
156   const showEvent =
157     Platform.OS === 'ios' ? 'keyboardWillShow' : 'keyboardDidShow';
158   const hideEvent =
159     Platform.OS === 'ios' ? 'keyboardWillHide' : 'keyboardDidHide';
160
161   const showSub = Keyboard.addListener(showEvent, (e: KeyboardEvent) => {
162     // translateY 为负 = 向上移动, 减去 BAR_BOTTOM_SPACING 留出间距
163     const targetY = -(e.endCoordinates.height + BAR_BOTTOM_SPACING);
164
165     Animated.parallel([
166       Animated.timing(translateY, {
167         toValue: targetY,
168         duration: e.duration || 280,
169         useNativeDriver: true,
170       }),
171       Animated.timing(opacity, {
172         toValue: 1,
173         duration: (e.duration || 280) * 0.5,
174         useNativeDriver: true,
175       }),
176     ]).start();
177   });
178
179   const hideSub = Keyboard.addListener(hideEvent, (e: KeyboardEvent) => {
180     Animated.parallel([
181       Animated.timing(translateY, {
182         toValue: 0,
183         duration: e.duration || 280,
184         useNativeDriver: true,
185       }),
186       Animated.timing(opacity, {
187         toValue: 0,
188         duration: (e.duration || 280) * 0.6,
189         useNativeDriver: true,
190       }),
191     ]).start();
192   });
193
194   return () => {
195     showSub.remove();
196     hideSub.remove();
197   };
198 }, [translateY, opacity]);

```

```

199
200 // ---- 收起 ----
201 const handleDone = () => {
202   onDismiss();
203 };
204
205 if (!activeField) return null;
206
207 return (
208   <Animated.View
209     style={[
210       styles.container,
211       {
212         backgroundColor: colors.backgroundDark,
213         borderTopColor: colors.borderLight,
214       },
215       {
216         transform: [{ translateY }],
217         opacity,
218       },
219     ]}
220   >
221     /* 输入行 */
222     <View
223       style={[
224         styles.inputRow,
225         {
226           backgroundColor: colors.background,
227           borderColor: colors.borderLight,
228         },
229       ]}
230     >
231       <Ionicons
232         name={activeField.icon as any}
233         size={18}
234         color={colors.primary}
235         style={styles.inputIcon}
236       />
237       <TextInput
238         key={activeFieldId}
239         ref={inputRef}
240         style={[styles.textInput, { color: colors.textPrimary }]}
241         value={activeField.value}
242         onChangeText={activeField.onChangeText}
243         placeholder={activeField.placeholder}
244         placeholderTextColor={colors.textSecondary}
245         keyboardType={activeField.keyboardType || 'default'}
246         secureTextEntry={activeField.secureTextEntry}
247         autoCapitalize="none"
248         autoCorrect={false}

```

```

249     autoFocus
250     returnKeyType="done"
251     onSubmitEditing={handleDone}
252   />
253   {activeField.value.length > 0 && (
254     <TouchableOpacity
255       onPress={() => activeField.onChangeText("")}
256       style={styles.clearButton}
257       hitSlop={{ top: 8, bottom: 8, left: 8, right: 8 }}
258     >
259       <Ionicons name="close-circle" size={18} color={colors.textSecondary} />
260     </TouchableOpacity>
261   )}
262   <View style={styles.doneDivider} />
263   <TouchableOpacity onPress={handleDone} style={styles.doneButton}>
264     <Text style={[styles.doneText, { color: colors.primary }]}>完成</Text>
265   </TouchableOpacity>
266 </View>
267 </Animated.View>
268 );
269 };
270
271 const styles = StyleSheet.create({
272   container: {
273     position: 'absolute',
274     bottom: 0,
275     left: 0,
276     right: 0,
277     borderTopWidth: StyleSheet.hairlineWidth,
278     paddingVertical: 10,
279     paddingHorizontal: 12,
280   },
281   inputRow: {
282     flexDirection: 'row',
283     alignItems: 'center',
284     borderRadius: 14,
285     borderWidth: 1,
286     paddingHorizontal: 14,
287     height: 48,
288   },
289   inputIcon: {
290     marginRight: 10,
291   },
292   textInput: {
293     flex: 1,
294     fontSize: 16,
295     paddingVertical: 0,
296   },
297   clearButton: {
298     padding: 4,

```

```

299   marginLeft: 4,
300 },
301 doneDivider: {
302   width: 1,
303   height: 20,
304   backgroundColor: 'rgba(128,128,128,0.25)',
305   marginHorizontal: 10,
306 },
307 doneButton: {
308   paddingVertical: 6,
309   paddingHorizontal: 4,
310   borderRadius: 8,
311 },
312 doneText: {
313   fontSize: 15,
314   fontWeight: '700',
315 },
316 });
317
318 === mobile/src/components/GameHelpModal.tsx ===
319 import React, { useMemo } from 'react';
320 import { View, Text, StyleSheet, Modal, TouchableOpacity, ScrollView } from 'react-native';
321 import { Ionicons } from '@expo/vector-icons';
322 import { useTheme } from '../contexts/ThemeContext';
323
324 interface GameHelpModalProps {
325   visible: boolean;
326   onClose: () => void;
327 }
328
329 const GameHelpModal: React.FC<GameHelpModalProps> = ({ visible, onClose }) => {
330   const { colors } = useTheme();
331
332   const styles = useMemo(() => StyleSheet.create({
333     container: {
334       flex: 1,
335       backgroundColor: colors.background,
336     },
337     header: {
338       flexDirection: 'row',
339       justifyContent: 'space-between',
340       alignItems: 'center',
341       padding: 16,
342       paddingTop: 48,
343       borderBottomWidth: 1,
344       borderBottomColor: colors.borderDark,
345     },
346     closeBtn: { padding: 8 },
347     title: {
348       color: colors.textPrimary,

```

```

348     fontSize: 20,
349     fontWeight: '700',
350   },
351   placeholder: { width: 44 },
352   content: { padding: 16 },
353   section: {
354     marginBottom: 24,
355     backgroundColor: colors.borderLight,
356     borderRadius: 12,
357     padding: 16,
358     borderWidth: 1,
359     borderColor: colors.border,
360   },
361   sectionTitle: {
362     color: colors.warning,
363     fontSize: 16,
364     fontWeight: '700',
365     marginBottom: 8,
366   },
367   sectionDesc: {
368     color: colors.textSecondary,
369     fontSize: 14,
370     lineHeight: 22,
371   },
372   rulesList: { gap: 6 },
373   ruleItem: {
374     color: colors.textSecondary,
375     fontSize: 14,
376     lineHeight: 22,
377   },
378  )), [colors]);
379
380 return (
381   <Modal
382     visible={visible}
383     animationType="slide"
384     onRequestClose={onClose}
385   >
386     <View style={styles.container}>
387       <View style={styles.header}>
388         <TouchableOpacity onPress={onClose} style={styles.closeBtn}>
389           <Ionicons name="close" size={28} color={colors.textSecondary} />
390         </TouchableOpacity>
391         <Text style={styles.title}>游戏说明</Text>
392         <View style={styles.placeholder} />
393       </View>
394
395       <ScrollView style={styles.content} showsVerticalScrollIndicator={false}>
396         <View style={styles.section}>
397           <Text style={styles.sectionTitle}> 游戏恢复体力 </Text>

```



```

398     <Text style={styles.sectionDesc}>
399         每天可游玩3次小游戏恢复体力，完成游戏后可获得体力和能量奖励。
400     </Text>
401 </View>
402
403 <View style={styles.section}>
404     <Text style={styles.sectionTitle}> 数独</Text>
405     <Text style={styles.sectionDesc}>
406         在9×9的网格中填入数字1-9，使每行、每列、每个3×3宫格内的数字都不重复。完成一局即可获得奖励！
407     </Text>
408 </View>
409
410 <View style={styles.section}>
411     <Text style={styles.sectionTitle}> 推箱子</Text>
412     <Text style={styles.sectionDesc}>
413         将所有箱子推到目标位置上即可过关。完成3关即可获得奖励！
414     </Text>
415 </View>
416
417 <View style={styles.section}>
418     <Text style={styles.sectionTitle}> 游戏规则</Text>
419     <View style={styles.rulesList}>
420         <Text style={styles.ruleItem}>• 数独：完成一局即可获得奖励</Text>
421         <Text style={styles.ruleItem}>• 推箱子：完成3关即可获得奖励</Text>
422         <Text style={styles.ruleItem}>• 每天可游玩3次，凌晨5点重置次数</Text>
423         <Text style={styles.ruleItem}>• 完成游戏后奖励会自动添加到你的账户</Text>
424         <Text style={styles.ruleItem}>• 游戏过程中可随时退出，不会消耗次数</Text>
425     </View>
426 </View>
427 </ScrollView>
428 </View>
429 </Modal>
430 );
431 };
432
433 export default GameHelpModal;
434
435 === mobile/src/components/GameModal.tsx ===
436 import React, { useState, useEffect, useMemo } from 'react';
437 import { View, Text, StyleSheet, TouchableOpacity, ScrollView, Alert, Modal } from 'react-native';
438 import { Ionicons } from '@expo/vector-icons';
439 import { useTheme } from '../contexts/ThemeContext';
440 import {
441     sokobanLevels,
442     getLevelByIndex,
443     calculateRewards,
444     mockSubmitGameResult,
445     type GameResult,
446     } from '../services/gameService';

```

```

447 // ===== 类型定义 =====
448
449 /**
450  * 游戏弹窗组件属性
451  */
452 interface GameModalProps {
453   /** 是否显示弹窗 */
454   visible: boolean;
455   /** 关闭弹窗回调 */
456   onClose: () => void;
457   /** 游戏完成回调, 返回奖励 */
458   onGameComplete: (rewards: { stamina: number; energy: number }) => void;
459   /** 怪兽类型, 影响奖励计算 */
460   monsterType?: string;
461 }
462
463 /**
464  * 当前显示屏幕类型
465  */
466 type Screen = 'welcome' | 'tutorial' | 'game';
467
468 /**
469  * 游戏类型
470  */
471 type GameType = 'sudoku' | 'sokoban';
472
473 // ===== 数独游戏组件 =====
474
475 /**
476  * 数独游戏组件
477  */
478 const SudokuGame = ({ onWin, colors }: { onWin: () => void; colors: any }) => {
479   /** 游戏面板, 0表示空 */
480   const [board, setBoard] = useState<number[][]>([]);
481   /** 答案面板 */
482   const [solution, setSolution] = useState<number[][]>([]);
483   /** 哪些位置是初始给定的 (不可修改) */
484   const [given, setGiven] = useState<boolean[][]>([]);
485   /** 当前选中的格子位置 */
486   const [selected, setSelected] = useState<{ row: number; col: number } | null>(null);
487
488   /**
489    * 检查数字在当前位置是否有效
490    */
491   const isValid = (grid: number[][], row: number, col: number, num: number): boolean => {
492     // 检查行
493     for (let i = 0; i < 9; i++) {
494       if (grid[row][i] === num || grid[i][col] === num) return false;
495     }
496     // 检查3x3宫格

```

```

497     const boxRow = Math.floor(row / 3) * 3;
498     const boxCol = Math.floor(col / 3) * 3;
499     for (let i = 0; i < 3; i++) {
500         for (let j = 0; j < 3; j++) {
501             if (grid[boxRow + i][boxCol + j] === num) return false;
502         }
503     }
504     return true;
505 };
506
507 /**
508  * 使用回溯法生成数独答案
509  */
510 const solveSudoku = (grid: number[][]): boolean => {
511     for (let row = 0; row < 9; row++) {
512         for (let col = 0; col < 9; col++) {
513             if (grid[row][col] === 0) {
514                 // 随机打乱数字1-9
515                 const nums = [1, 2, 3, 4, 5, 6, 7, 8, 9].sort(() => Math.random() - 0.5);
516                 for (const num of nums) {
517                     if (isValid(grid, row, col, num)) {
518                         grid[row][col] = num;
519                         if (solveSudoku(grid)) return true;
520                         grid[row][col] = 0;
521                     }
522                 }
523                 return false;
524             }
525         }
526     }
527     return true;
528 };
529
530 /**
531  * 初始化游戏
532  */
533 const initGame = () => {
534     // 生成答案
535     const newSolution = Array(9).fill(0).map(() => Array(9).fill(0));
536     solveSudoku(newSolution);
537
538     // 复制答案到面板，然后挖空一些格子
539     const newBoard = newSolution.map(row => [...row]);
540     const newGiven = Array(9).fill(0).map(() => Array(9).fill(false));
541
542     // 随机挖空35个格子
543     const cellsToRemove = 35;
544     let removed = 0;
545     while (removed < cellsToRemove) {
546         const row = Math.floor(Math.random() * 9);

```

```

547     const col = Math.floor(Math.random() * 9);
548     if (newBoard[row][col] !== 0) {
549         newBoard[row][col] = 0;
550         removed++;
551     }
552 }
553
554 // 标记哪些是给定的
555 for (let row = 0; row < 9; row++) {
556     for (let col = 0; col < 9; col++) {
557         newGiven[row][col] = newSolution[row][col] === newBoard[row][col] && newBoard[row][col] !== 0;
558     }
559 }
560
561 setSolution(newSolution);
562 setBoard(newBoard);
563 setGiven(newGiven);
564 setSelected(null);
565 };
566
567 // 组件挂载时初始化游戏
568 useEffect(() => {
569     initGame();
570 }, []);
571
572 /**
573  * 检查游戏是否胜利
574  */
575 const checkWin = (): boolean => {
576     for (let row = 0; row < 9; row++) {
577         for (let col = 0; col < 9; col++) {
578             if (board[row][col] !== solution[row][col]) return false;
579         }
580     }
581     return true;
582 };
583
584 /**
585  * 输入数字处理
586  */
587 const handleInput = (num: number) => {
588     if (!selected || given[selected.row][selected.col]) return;
589
590     const newBoard = board.map(row => [...row]);
591     newBoard[selected.row][selected.col] = num;
592     setBoard(newBoard);
593
594     // 检查胜利
595     if (checkWin()) {
596         setTimeout(() => onWin(), 300);

```

```

597   }
598 };
599
600 /**
601  * 清除数字
602  */
603 const handleClear = () => {
604   if (!selected || given[selected.row][selected.col]) return;
605
606   const newBoard = board.map(row => [...row]);
607   newBoard[selected.row][selected.col] = 0;
608   setBoard(newBoard);
609 };
610
611 /**
612  * 获取格子背景色
613  */
614 const getCellColor = (row: number, col: number): string => {
615   if (!selected) return colors.background;
616   if (selected.row === row && selected.col === col) return colors.borderDark;
617   if (selected.row === row || selected.col === col) return colors.borderLight;
618   const boxRow = Math.floor(selected.row / 3) * 3;
619   const boxCol = Math.floor(selected.col / 3) * 3;
620   if (row >= boxRow && row < boxRow + 3 && col >= boxCol && col < boxCol + 3) {
621     return colors.borderLight;
622   }
623   return colors.background;
624 };
625
626 return (
627   <View style={styles.gameContainer}>
628     /* 头部 */
629     <View style={styles.sudokuHeader}>
630       <Text style={styles.sudokuTitle}> 数独</Text>
631       <TouchableOpacity style={styles.newGameBtn} onPress={initGame}>
632         <Ionicons name="refresh" size={16} color={colors.primary} />
633         <Text style={styles.newGameText}>新游戏</Text>
634       </TouchableOpacity>
635     </View>
636
637     /* 游戏面板 */
638     <View style={styles.sudokuGrid}>
639       {board.map((row, rowIndex) => (
640         <View key={rowIndex} style={styles.sudokuRow}>
641           {row.map((cell, colIndex) => (
642             <TouchableOpacity
643               key={colIndex}
644               style={[styles.sudokuCell, { backgroundColor: getCellColor(rowIndex, colIndex) }]}
645               onPress={() => !given[rowIndex][colIndex] && setSelected({ row: rowIndex, col: colIndex })}
646               disabled={given[rowIndex][colIndex]}

```

```

647         >
648         {cell !== 0 && (
649             <Text style=[
650                 styles.sudokuCellText,
651                 given[rowIndex][colIndex] ? styles.sudokuGiven : styles.sudokuInput
652             ]>
653             {cell}
654         </Text>
655     )}
656 </TouchableOpacity>
657 )))
658 </View>
659 )))
660 </View>
661
662 {/* 数字键盘 */}
663 <View style={styles.numberPad}>
664     {[1, 2, 3, 4, 5, 6, 7, 8, 9].map(num => (
665         <TouchableOpacity
666             key={num}
667             style={styles.numberBtn}
668             onPress={() => handleInput(num)}
669         >
670             <Text style={styles.numberBtnText}>{num}</Text>
671         </TouchableOpacity>
672     )]}
673     <TouchableOpacity style={styles.clearBtn} onPress={handleClear}>
674         <Text style={styles.clearBtnText}>清除</Text>
675     </TouchableOpacity>
676 </View>
677
678 </View>
679 );
680 };
681
682 // ===== 推箱子游戏组件 =====
683
684 /**
685  * 推箱子游戏组件
686  */
687 const SokobanGame = ({ onWin, monsterType, colors }: { onWin: (level: number) => void; monsterType?: string; c
688 /** 当前关卡索引 */
689 const [currentLevel, setCurrentLevel] = useState(0);
690 /** 玩家位置 */
691 const [playerPos, setPlayerPos] = useState({ x: 0, y: 0 });
692 /** 游戏地图 */
693 const [gameMap, setGameMap] = useState<string[][]>([]);
694
695 /**
696  * 初始化指定关卡

```

```

697  */
698  const initLevel = (levelIndex: number) => {
699    const level = getLevelByIndex(levelIndex);
700    const newMap = level.map.map(row => row.split(""));
701    let pPos = { x: 0, y: 0 };
702
703    // 找到玩家位置并将其从地图中移除
704    for (let y = 0; y < newMap.length; y++) {
705      for (let x = 0; x < newMap[y].length; x++) {
706        if (newMap[y][x] === '@') {
707          pPos = { x, y };
708          newMap[y][x] = ' ';
709        }
710      }
711    }
712
713    setCurrentLevel(levelIndex);
714    setGameMap(newMap);
715    setPlayerPos(pPos);
716  };
717
718  // 组件挂载时初始化第一关
719  useEffect(() => {
720    initLevel(0);
721  }, []);
722
723  /**
724   * 检查当前地图是否胜利
725   */
726  const checkWin = (map: string[][]): boolean => {
727    for (const row of map) {
728      if (row.includes("$") || row.includes('.')) return false;
729    }
730    return true;
731  };
732
733  /**
734   * 移动玩家
735   */
736  const move = (dx: number, dy: number) => {
737    // 重置当前关卡
738    if (dx === 0 && dy === 0) {
739      initLevel(currentLevel);
740      return;
741    }
742
743    const newX = playerPos.x + dx;
744    const newY = playerPos.y + dy;
745
746    // 检查边界

```

```

747 if (newY < 0 || newY >= gameMap.length || newX < 0 || newX >= gameMap[newY].length) return;
748
749 const target = gameMap[newY][newX];
750 if (target === '#') return; // 撞墙
751
752 const newMap = gameMap.map(row => [...row]);
753
754 // 推箱子
755 if (target === '$' || target === '*') {
756   const boxNewX = newX + dx;
757   const boxNewY = newY + dy;
758
759   // 检查箱子移动位置
760   if (boxNewY < 0 || boxNewY >= gameMap.length || boxNewX < 0 || boxNewX >= gameMap[boxNewY].length) retur
761
762   const boxTarget = gameMap[boxNewY][boxNewX];
763   if (boxTarget === '.' || boxTarget === '*') {
764     newMap[newY][newX] = target === '*' ? ' ': '.';
765     newMap[boxNewY][boxNewX] = boxTarget === '.' ? '*' : '$';
766   } else {
767     return;
768   }
769 }
770
771 setGameMap(newMap);
772 setPlayerPos({ x: newX, y: newY });
773
774 // 检查胜利
775 const currentLevelCopy = currentLevel;
776 if (checkWin(newMap)) {
777   setTimeout(() => {
778     if (currentLevelCopy < 2) {
779       initLevel(currentLevelCopy + 1);
780     } else {
781       onWin(currentLevelCopy + 1);
782     }
783   }, 500);
784 }
785 };
786
787 /**
788  * 获取格子内容和样式
789  */
790 const getCellContent = (char: string, x: number, y: number): { type: string; content: string } => {
791   if (playerPos.x === x && playerPos.y === y) {
792     return { type: 'player', content: '' };
793   }
794   switch (char) {
795     case '#': return { type: 'wall', content: '' };
796     case '$': return { type: 'box', content: '' };

```



```

797     case '*': return { type: 'box-on-target', content: '' };
798     case '.': return { type: 'target', content: '' };
799     default: return { type: 'floor', content: '' };
800 }
801 };
802
803 const level = getLevelByIndex(currentLevel);
804
805 return (
806   <View style={styles.gameContainer}>
807     /* 头部 */
808     <View style={styles.sokobanHeader}>
809       <Text style={styles.levelName}>{level.name} / {sokobanLevels.length}</Text>
810       <TouchableOpacity style={styles.newGameBtn} onPress={() => initLevel(0)}>
811         <Ionicons name="refresh" size={16} color={colors.primary} />
812         <Text style={styles.newGameText}>重新开始</Text>
813       </TouchableOpacity>
814     </View>
815
816     /* 游戏地图 */
817     <View style={styles.sokobanGrid}>
818       {gameMap.map((row, rowIndex) => (
819         <View key={rowIndex} style={styles.sokobanRow}>
820           {row.map((char, colIndex) => {
821             const { type, content } = getCellContent(char, colIndex, rowIndex);
822             const cellStyle = type === 'wall' ? styles.sokobanCellWall :
823               type === 'floor' ? styles.sokobanCellFloor :
824               type === 'target' ? styles.sokobanCellTarget :
825               type === 'player' ? styles.sokobanCellPlayer :
826               type === 'box' ? styles.sokobanCellBox :
827               styles.sokobanCellBoxOnTarget;
828             return (
829               <View key={colIndex} style={[styles.sokobanCell, cellStyle]}>
830                 <Text style={styles.sokobanCellText}>{content}</Text>
831               </View>
832             );
833           })}
834         </View>
835       ))}
836     </View>
837
838     /* 方向控制 */
839     <View style={styles.sokobanControls}>
840       <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(0, -1)}>
841         <Text style={styles.sokobanBtnText}></Text>
842       </TouchableOpacity>
843       <View style={styles.sokobanBtnRow}>
844         <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(-1, 0)}>
845           <Text style={styles.sokobanBtnText}></Text>
846         </TouchableOpacity>

```

```

847     <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(0, 0)}>
848       <Text style={styles.sokobanBtnText}></Text>
849     </TouchableOpacity>
850     <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(1, 0)}>
851       <Text style={styles.sokobanBtnText}></Text>
852     </TouchableOpacity>
853   </View>
854   <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(0, 1)}>
855     <Text style={styles.sokobanBtnText}></Text>
856   </TouchableOpacity>
857 </View>
858
859 </View>
860 );
861 };
862
863 // ===== 主游戏弹窗组件 =====
864
865 /**
866  * 游戏弹窗主组件
867  * 包含游戏选择、教程和游戏界面
868  */
869 const GameModal = ({ visible, onClose, onGameComplete, monsterType }: GameModalProps) => {
870   const { colors } = useTheme();
871
872   // ===== 样式 =====
873
874   const styles = useMemo(() => StyleSheet.create({
875     container: {
876       flex: 1,
877       backgroundColor: colors.background,
878     },
879     header: {
880       flexDirection: 'row',
881       justifyContent: 'space-between',
882       alignItems: 'center',
883       padding: 16,
884       paddingTop: 40,
885       borderBottomWidth: 1,
886       borderBottomColor: 'colors.borderDark',
887     },
888     closeBtn: {
889       padding: 8,
890     },
891     title: {
892       fontSize: 20,
893       fontWeight: 'bold',
894       color: colors.textPrimary,
895     },
896     placeholder: {

```

```

897   width: 44,
898 },
899 welcomeContent: {
900   flex: 1,
901   padding: 16,
902 },
903 welcomeSection: {
904   textAlign: 'center',
905   marginBottom: 24,
906 },
907 welcomeTitle: {
908   fontSize: 28,
909   fontWeight: 'bold',
910   color: colors.primary,
911   marginBottom: 8,
912 },
913 welcomeSubtitle: {
914   fontSize: 14,
915   color: colors.textSecondary,
916 },
917 sectionTitle: {
918   fontSize: 16,
919   fontWeight: 'bold',
920   color: colors.warning,
921   marginBottom: 12,
922 },
923 rewardsSection: {
924   backgroundColor: 'rgba(255,215,0,0.08)',
925   borderRadius: 12,
926   padding: 16,
927   marginBottom: 16,
928 },
929 rewardsRow: {
930   flexDirection: 'row',
931   justifyContent: 'center',
932   gap: 40,
933 },
934 rewardBox: {
935   alignItems: 'center',
936 },
937 rewardIcon: {
938   fontSize: 32,
939   marginBottom: 4,
940 },
941 rewardAmount: {
942   fontSize: 20,
943   fontWeight: 'bold',
944   color: colors.warning,
945 },
946 rewardName: {

```

```

947   fontSize: 12,
948   color: colors.textSecondary,
949 },
950 rewardNote: {
951   fontSize: 12,
952   color: colors.textSecondary,
953   textAlign: 'center',
954   marginTop: 12,
955 },
956 gamesSection: {
957   marginBottom: 16,
958 },
959 gameCard: {
960   flexDirection: 'row',
961   alignItems: 'center',
962   backgroundColor: colors.surface,
963   borderRadius: 12,
964   padding: 16,
965   marginBottom: 12,
966   borderWidth: 1,
967   borderColor: 'colors.borderDark',
968 },
969 gameCardIcon: {
970   fontSize: 36,
971   marginRight: 12,
972 },
973 gameCardInfo: {
974   flex: 1,
975 },
976 gameCardTitle: {
977   fontSize: 16,
978   fontWeight: 'bold',
979   color: colors.textPrimary,
980   marginBottom: 4,
981 },
982 gameCardDesc: {
983   fontSize: 12,
984   color: colors.textSecondary,
985 },
986 gameCardArrow: {
987   fontSize: 20,
988   color: colors.primary,
989 },
990 tipsSection: {
991   backgroundColor: 'rgba(123,117,216,0.08)',
992   borderRadius: 12,
993   padding: 16,
994 },
995 tipsText: {
996   fontSize: 12,

```

```

997     color: colors.textSecondary,
998     lineHeight: 20,
999   },
1000   tipsList: {
1001     gap: 6,
1002   },
1003   tiplItem: {
1004     fontSize: 12,
1005     color: colors.textSecondary,
1006     lineHeight: 20,
1007   },
1008   tutorialContent: {
1009     flex: 1,
1010     padding: 16,
1011   },
1012   backBtn: {
1013     flexDirection: 'row',
1014     alignItems: 'center',
1015     gap: 4,
1016     marginBottom: 16,
1017     zIndex: 200,
1018     position: 'relative',
1019   },
1020   backBtnText: {
1021     color: colors.textSecondary,
1022     fontSize: 14,
1023   },
1024   tutorialHeader: {
1025     flexDirection: 'row',
1026     alignItems: 'center',
1027     gap: 12,
1028     marginBottom: 20,
1029   },
1030   tutorialIcon: {
1031     fontSize: 40,
1032   },
1033   tutorialTitle: {
1034     fontSize: 22,
1035     fontWeight: 'bold',
1036     color: colors.textPrimary,
1037   },
1038   tutorialSection: {
1039     marginBottom: 24,
1040     paddingBottom: 8,
1041   },
1042   tutorialSectionTitle: {
1043     fontSize: 16,
1044     fontWeight: 'bold',
1045     color: colors.warning,
1046     marginBottom: 8,

```

```

1047 },
1048 tutorialText: {
1049   fontSize: 14,
1050   color: colors.textSecondary,
1051   lineHeight: 22,
1052 },
1053 tutorialSteps: {
1054   gap: 6,
1055 },
1056 tutorialStep: {
1057   fontSize: 14,
1058   color: colors.textSecondary,
1059   lineHeight: 22,
1060 },
1061 startBtn: {
1062   backgroundColor: colors.primary,
1063   borderRadius: 12,
1064   padding: 16,
1065   alignItems: 'center',
1066   marginTop: 32,
1067   marginBottom: 32,
1068 },
1069 startBtnText: {
1070   fontSize: 16,
1071   fontWeight: 'bold',
1072   color: '#FFFFFF',
1073 },
1074 gameScreen: {
1075   flex: 1,
1076   padding: 16,
1077   paddingTop: 40,
1078 },
1079 gameContent: {
1080   flex: 1,
1081 },
1082 gameContainer: {
1083   flex: 1,
1084 },
1085 // 数独样式
1086 sudokuHeader: {
1087   flexDirection: 'row',
1088   justifyContent: 'space-between',
1089   alignItems: 'center',
1090   marginBottom: 16,
1091 },
1092 sudokuTitle: {
1093   fontSize: 18,
1094   fontWeight: 'bold',
1095   color: colors.textPrimary,
1096 },

```

```

1097 newGameBtn: {
1098   flexDirection: 'row',
1099   alignItems: 'center',
1100   gap: 4,
1101   padding: 8,
1102   backgroundColor: 'colors.borderLight',
1103   borderRadius: 8,
1104 },
1105 newGameText: {
1106   fontSize: 12,
1107   color: colors.primary,
1108 },
1109 sudokuGrid: {
1110   backgroundColor: colors.backgroundDark,
1111   borderRadius: 8,
1112   padding: 4,
1113   marginBottom: 16,
1114   borderWidth: 2,
1115   borderColor: colors.primary,
1116 },
1117 sudokuRow: {
1118   flexDirection: 'row',
1119 },
1120 sudokuCell: {
1121   flex: 1,
1122   aspectRatio: 1,
1123   alignItems: 'center',
1124   justifyContent: 'center',
1125   borderWidth: 1,
1126   borderColor: 'colors.borderDark',
1127 },
1128 sudokuCellText: {
1129   fontSize: 18,
1130   fontWeight: 'bold',
1131 },
1132 sudokuGiven: {
1133   color: colors.primary,
1134 },
1135 sudokuInput: {
1136   color: colors.warning,
1137 },
1138 sudokuSolution: {
1139   color: colors.success,
1140 },
1141 numberPad: {
1142   flexDirection: 'row',
1143   flexWrap: 'wrap',
1144   gap: 6,
1145   marginBottom: 16,
1146 },

```

```

1147   numberBtn: {
1148     width: '18%',
1149     aspectRatio: 1,
1150     backgroundColor: 'colors.borderDark',
1151     borderRadius: 8,
1152     alignItems: 'center',
1153     justifyContent: 'center',
1154   },
1155   numberBtnText: {
1156     fontSize: 18,
1157     fontWeight: 'bold',
1158     color: colors.primary,
1159   },
1160   clearBtn: {
1161     width: '18%',
1162     aspectRatio: 1,
1163     backgroundColor: 'rgba(255,107,107,0.2)',
1164     borderRadius: 8,
1165     alignItems: 'center',
1166     justifyContent: 'center',
1167   },
1168   clearBtnText: {
1169     fontSize: 12,
1170     fontWeight: 'bold',
1171     color: '#FF6B6B',
1172   },
1173   hintBtn: {
1174     backgroundColor: 'rgba(255,215,0,0.15)',
1175     borderRadius: 10,
1176     padding: 14,
1177     alignItems: 'center',
1178     borderWidth: 1,
1179     borderColor: 'rgba(255,215,0,0.4)',
1180   },
1181   hintBtnText: {
1182     fontSize: 14,
1183     fontWeight: 'bold',
1184     color: colors.warning,
1185   },
1186   // 推箱子样式
1187   sokobanHeader: {
1188     flexDirection: 'row',
1189     justifyContent: 'space-between',
1190     alignItems: 'center',
1191     marginBottom: 12,
1192   },
1193   levelName: {
1194     fontSize: 16,
1195     fontWeight: 'bold',
1196     color: colors.warning,

```



```

1197 },
1198 sokobanGrid: {
1199   backgroundColor: colors.backgroundDark,
1200   borderRadius: 8,
1201   padding: 8,
1202   marginBottom: 16,
1203   borderWidth: 2,
1204   borderColor: colors.orange,
1205 },
1206 sokobanRow: {
1207   flexDirection: 'row',
1208   justifyContent: 'center',
1209 },
1210 sokobanCell: {
1211   width: 40,
1212   height: 40,
1213   alignItems: 'center',
1214   justifyContent: 'center',
1215   borderRadius: 4,
1216   margin: 1,
1217 },
1218 sokobanCellWall: {
1219   backgroundColor: '#3A3A5C',
1220 },
1221 sokobanCellFloor: {
1222   backgroundColor: colors.background,
1223 },
1224 sokobanCellTarget: {
1225   backgroundColor: 'rgba(255,215,0,0.2)',
1226 },
1227 sokobanCellPlayer: {
1228   backgroundColor: 'colors.borderDark',
1229 },
1230 sokobanCellBox: {
1231   backgroundColor: 'rgba(255,125,0,0.4)',
1232 },
1233 sokobanCellBoxOnTarget: {
1234   backgroundColor: 'rgba(58,227,116,0.4)',
1235 },
1236 sokobanCellText: {
1237   fontSize: 20,
1238 },
1239 sokobanControls: {
1240   alignItems: 'center',
1241   gap: 4,
1242   marginBottom: 16,
1243 },
1244 sokobanBtnRow: {
1245   flexDirection: 'row',
1246   gap: 4,

```

```

1247 },
1248 sokobanBtn: {
1249   width: 55,
1250   height: 55,
1251   backgroundColor: 'colors.borderDark',
1252   borderRadius: 10,
1253   alignItems: 'center',
1254   justifyContent: 'center',
1255   borderWidth: 1,
1256   borderColor: 'colors.borderDark',
1257 },
1258 sokobanBtnText: {
1259   fontSize: 22,
1260 },
1261 answerContainer: {
1262   backgroundColor: 'rgba(255,215,0,0.08)',
1263   borderRadius: 10,
1264   padding: 12,
1265   marginBottom: 12,
1266   borderWidth: 1,
1267   borderColor: 'rgba(255,215,0,0.25)',
1268 },
1269 answerTitle: {
1270   fontSize: 13,
1271   fontWeight: 'bold',
1272   color: colors.warning,
1273   marginBottom: 8,
1274 },
1275 answerSteps: {
1276   flexDirection: 'row',
1277   flexWrap: 'wrap',
1278   gap: 8,
1279   marginBottom: 8,
1280 },
1281 stepPending: {
1282   fontSize: 14,
1283   color: colors.textSecondary,
1284 },
1285 stepDone: {
1286   fontSize: 14,
1287   color: colors.success,
1288 },
1289 answerButtons: {
1290   flexDirection: 'row',
1291   gap: 8,
1292 },
1293 smallBtn: {
1294   flex: 1,
1295   padding: 8,
1296   backgroundColor: 'colors.borderDark',

```

```

1297   borderRadius: 6,
1298   alignItems: 'center',
1299 },
1300 smallBtnText: {
1301   fontSize: 11,
1302   color: colors.primary,
1303   fontWeight: 'bold',
1304 },
1305 winOverlay: {
1306   position: 'absolute',
1307   top: 0,
1308   left: 0,
1309   right: 0,
1310   bottom: 0,
1311   backgroundColor: 'rgba(0,0,0,0.7)',
1312   alignItems: 'center',
1313   justifyContent: 'center',
1314   zIndex: 100,
1315   pointerEvents: 'none',
1316 },
1317 winText: {
1318   fontSize: 32,
1319   fontWeight: 'bold',
1320   color: colors.warning,
1321 },
1322 submittingText: {
1323   fontSize: 14,
1324   color: colors.textSecondary,
1325   marginTop: 16,
1326 },
1327 )), [colors]);
1328  /** 当前显示的屏幕 */
1329  const [currentScreen, setCurrentScreen] = useState<Screen>('welcome');
1330  /** 当前选中的游戏类型 */
1331  const [currentGame, setCurrentGame] = useState<GameType>('sudoku');
1332  /** 是否游戏胜利 */
1333  const [gameWon, setGameWon] = useState(false);
1334  /** 是否正在提交结果 */
1335  const [isSubmitting, setIsSubmitting] = useState(false);
1336
1337  /**
1338   * 处理游戏胜利
1339   */
1340  const handleWin = async (level?: number) => {
1341    console.log('[GameModal] 游戏胜利 - 关卡:', level);
1342    setGameWon(true);
1343    setIsSubmitting(true);
1344
1345    try {
1346      const result: GameResult = await mockSubmitGameResult(

```

```

1347     level || 1,
1348     true,
1349     monsterType
1350   );
1351
1352   if (result.success) {
1353     console.log('[GameModal] 游戏结果提交成功 - 奖励:', result.rewards);
1354     setTimeout(() => {
1355       setGameWon(false);
1356       setIsSubmitting(false);
1357       onGameComplete(result.rewards);
1358     }, 500);
1359   } else {
1360     throw new Error(result.message || '游戏提交失败');
1361   }
1362 } catch (error) {
1363   console.error('[GameModal] 游戏提交失败:', error);
1364   setIsSubmitting(false);
1365   setGameWon(false);
1366   Alert.alert('提交失败', '游戏结果提交失败, 请重试', [
1367     { text: '重试', onPress: () => handleWin(level) },
1368     { text: '取消', style: 'cancel' }
1369   ]);
1370 }
1371 };
1372
1373 /**
1374  * 显示游戏教程
1375  */
1376 const showTutorial = (gameType: GameType) => {
1377   setCurrentGame(gameType);
1378   setCurrentScreen('tutorial');
1379 };
1380
1381 /**
1382  * 开始游戏
1383  */
1384 const startGame = () => {
1385   setCurrentScreen('game');
1386   setGameWon(false);
1387 };
1388
1389 /**
1390  * 返回上一步
1391  */
1392 const goBack = () => {
1393   if (currentScreen === 'game') {
1394     Alert.alert('确认退出', '退出后当前游戏进度将丢失, 确定退出吗? ', [
1395       { text: '取消', style: 'cancel' },
1396       { text: '确定', onPress: () => {

```

```

1397     setCurrentScreen('tutorial');
1398     setGameWon(false);
1399   }}
1400   });
1401   } else {
1402     setCurrentScreen('welcome');
1403     setGameWon(false);
1404   }
1405   };
1406
1407   /**
1408    * 关闭弹窗
1409    */
1410   const handleClose = () => {
1411     if (currentScreen === 'game') {
1412       Alert.alert(' 确认退出', '退出后当前游戏进度将丢失， 确定退出吗? ', [
1413         { text: '取消', style: 'cancel' },
1414         { text: '确定', onPress: () => {
1415           setCurrentScreen('welcome');
1416           setGameWon(false);
1417           onClose();
1418         }}
1419       ]);
1420     } else {
1421       setCurrentScreen('welcome');
1422       setGameWon(false);
1423       onClose();
1424     }
1425   };
1426
1427   return (
1428     <Modal
1429       visible={visible}
1430       animationType="slide"
1431       onRequestClose={handleClose}
1432     >
1433     <View style={styles.container}>
1434       {/* 头部栏 */}
1435       <View style={styles.header}>
1436         <TouchableOpacity onPress={handleClose} style={styles.closeBtn}>
1437           <Ionicons name="close" size={28} color={colors.textSecondary} />
1438         </TouchableOpacity>
1439         <Text style={styles.title}>游戏恢复</Text>
1440         <View style={styles.placeholder} />
1441       </View>
1442
1443       {/* 欢迎页面 */}
1444       {currentScreen === 'welcome' && (
1445         <ScrollView style={styles.welcomeContent} showsVerticalScrollIndicator={false}>
1446         <View style={styles.welcomeSection}>

```

```

1447     <Text style={styles.welcomeTitle}> 游戏恢复</Text>
1448     <Text style={styles.welcomeSubtitle}>完成小游戏，恢复体力和能量！</Text>
1449   </View>
1450
1451   {/* 奖励说明 */}
1452   <View style={styles.rewardsSection}>
1453     <Text style={styles.sectionTitle}> 游戏奖励</Text>
1454     <View style={styles.rewardsRow}>
1455       <View style={styles.rewardBox}>
1456         <Text style={styles.rewardIcon}> </Text>
1457         <Text style={styles.rewardAmount}> +5</Text>
1458         <Text style={styles.rewardName}>体力</Text>
1459       </View>
1460       <View style={styles.rewardBox}>
1461         <Text style={styles.rewardIcon}> </Text>
1462         <Text style={styles.rewardAmount}> +3</Text>
1463         <Text style={styles.rewardName}>能量</Text>
1464       </View>
1465     </View>
1466     <Text style={styles.rewardNote}> 叛逆小怪可获得双倍奖励！</Text>
1467   </View>
1468
1469   {/* 游戏选择 */}
1470   <View style={styles.gamesSection}>
1471     <Text style={styles.sectionTitle}> 选择游戏</Text>
1472
1473     <TouchableOpacity
1474       style={styles.gameCard}
1475       onPress={() => showTutorial('sudoku')}
1476     >
1477       <View style={styles.gameCardIcon}> </View>
1478       <View style={styles.gameCardInfo}>
1479         <Text style={styles.gameCardTitle}>数独</Text>
1480         <Text style={styles.gameCardDesc}>
1481           填写9×9网格，使每行、每列、每个3×3宫格都包含1-9不重复的数字。完成一局即可获得奖励！
1482         </Text>
1483       </View>
1484       <Text style={styles.gameCardArrow}>→</Text>
1485     </TouchableOpacity>
1486
1487     <TouchableOpacity
1488       style={styles.gameCard}
1489       onPress={() => showTutorial('sokoban')}
1490     >
1491       <View style={styles.gameCardIcon}> </View>
1492       <View style={styles.gameCardInfo}>
1493         <Text style={styles.gameCardTitle}>推箱子</Text>
1494         <Text style={styles.gameCardDesc}>
1495           将所有箱子推到目标位置上即可过关。完成3关即可获得奖励！
1496         </Text>

```